

HW1 — Optimizing a 1×1 Convolution through Data Layout

1. General approach and thought process

I chose the **1×1 convolution** (a.k.a. *pointwise convolution*), the workhorse layer of modern CNNs (MobileNet, ResNet bottlenecks, etc.). It is appealing for this assignment because it is *arithmetically trivial* — just a weighted sum across channels — yet its performance is **entirely dominated by how the data is laid out in memory**. That makes it a clean vehicle for showing the hardware/software interplay: the math is fixed, so any speedup must come from cache behaviour, prefetching, and vectorization rather than from doing less work.

A 1×1 conv maps an input feature map `[Cin, H, W]` to `[Cout, H, W]` using weights `[Cout, Cin]`:

$$\text{out}[\text{co}][\text{h}][\text{w}] = \sum_{\text{ci}} \text{in}[\text{ci}][\text{h}][\text{w}] \cdot \text{weight}[\text{co}][\text{ci}]$$

For every output pixel this is a dot product over the input channels. My plan was: write the textbook version in the standard **NCHW** layout, profile it, identify the memory bottleneck, and then attack *only that bottleneck* while keeping the output bit-identical.

Both versions are compiled with the **same flags** (`-O3 -mavx2 -mfma`) so the measured difference is attributable to the optimization, **not** to the compiler. Inputs are generated by an identical deterministic function in both programs, and each prints a layout-independent checksum + sample values so correctness can be verified with a simple `diff` (confirmed **identical**).

Problem size: `Cin = Cout = 512`, `H = W = 128`. This is deliberate: the input and output tensors are ~32 MB each, which **exceeds the L3 cache**. If the data fit in cache, the strided-access penalty would be hidden and the optimization would look pointless — the effect only becomes visible (and meaningful) once the working set spills to DRAM.

Measurement environment. Profiling was done with `perf stat` on an Ubuntu x86-64 host. The machine is virtualized, so the PMU exposes only a subset of counters: `cycles`, `LLC-*`, and `branch-misses` came back as `0 / <not supported>`. The **L1 data-cache** counters *are* available, and since our hypothesis is precisely about L1 spatial locality, they tell the whole story; we use wall-clock time and retired-instruction throughput in place of the unavailable cycle counter.

2. Unoptimized implementation — [conv1x1_naive.cpp](#)

Why I picked it. It is the most natural way a student writes the layer: standard NCHW layout, loop nest `(co, h, w, ci)`, accumulate, store. It is correct and readable — and quietly cache-hostile.

What it does. Allocates the input (NCHW), weights (`[Cout][Cin]`), and output (NCHW), fills them deterministically, then computes the convolution with the four nested loops, and prints a checksum.

Why it is not optimized. The reduction loop runs over the input channel `ci`, but in NCHW two consecutive channels of the *same pixel* are `H*W = 16384` floats — **64 KB** — apart in memory:

```
for (int ci = 0; ci < Cin; ++ci)
    acc += in[ci*HW + h*W + w] * wt[co*Cin + ci]; // in[] strides by 64 KB each step
```

Consequences at the hardware level:

- **Spatial locality is destroyed.** Each `ci` step touches a different cache line; most of the 64-byte line that gets fetched is never used before it is evicted.
- **The hardware prefetcher cannot help** — it locks onto unit-stride streams, not 64 KB jumps.
- **No vectorization** — the strided load forces scalar code (or an expensive gather).
- **No reuse** — the input is re-streamed from memory once per output channel (`Cout` times).

Profiling results (naive):

Metric	Value
Runtime (compute)	11.37 s
Instructions	4.459×10^9
L1-dcache loads	1.650×10^9
L1-dcache load misses	1.587×10^9 — 96.17 % miss rate
Instruction throughput	≈ 0.38 G instr/s
Cycles / LLC / branch-misses	<not supported> (virtualized PMU)

The 96 % L1 miss rate is the smoking gun: almost **every** input load misses the L1 cache and must be served from a lower level, so the CPU spends most of its time stalled on memory rather than computing — only ~0.38 billion instructions retire per second.

3. Optimization — [conv1x1_opt.cpp](#)

What I changed. Two coupled changes, same math, identical output:

1. **Data layout NCHW → NHWC.** I transpose the input so that, for a fixed pixel, the `Cin` channel values are **contiguous**. The reduction over `ci` becomes a **unit-stride sweep**; the weight row `weight[co][.]` is contiguous over `ci` as well.
2. **Register blocking over output channels (4 at a time).** Each loaded input value `x` is reused across 4 output-channel accumulators held in registers.

```
const float* xin = &in_nhwc[p*Cin];          // Cin contiguous floats
for (int co = 0; co < Cout; co += 4) {
    float a0=0,a1=0,a2=0,a3=0;
    for (int ci = 0; ci < Cin; ++ci) {        // unit-stride, vectorizable
        float x = xin[ci];
        a0 += x*w0[ci]; a1 += x*w1[ci];
        a2 += x*w2[ci]; a3 += x*w3[ci];        // x reused 4x
    }
    /* store a0..a3 */
}
```

Why this specific optimization. The naive profile pointed straight at memory stalls, not at a shortage of compute. The fix therefore had to change *how data is fetched*, not how much arithmetic is done. Reshaping the per-pixel computation into a contiguous dot product turns the whole op into a cache- and SIMD-friendly GEMM — exactly how real inference engines and NN accelerators lay out pointwise convolutions.

System/hardware insight that led to it. Caches move data in 64-byte lines and the prefetcher rewards unit-stride access. A reduction whose stride is one cache line per step (naive) wastes ~15/16 of every line and defeats prefetching; making that same reduction contiguous lets one line serve 16 consecutive MACs, lets the prefetcher stream ahead, and lets the compiler emit packed FMA instructions. Register blocking then raises **arithmetic intensity** (FLOPs per byte loaded) so the execution units stay busy instead of stalling on memory → far higher retired-instruction throughput.

Profiling results (optimized):

Metric	Value
Runtime (compute)	2.28 s
Instructions	8.763×10^9
L1-dcache loads	5.404×10^9
L1-dcache load misses	0.295×10^9 — 5.45 % miss rate
Instruction throughput	≈ 3.67 G instr/s
Cycles / LLC / branch-misses	<not supported> (virtualized PMU)

Expected or surprising? The *direction* was expected: the L1 miss rate collapses from 96 % to 5.5 % and runtime drops ~5×. The **surprising** part is that the optimized version executes **more** instructions (8.76 B vs 4.46 B) and issues **more** L1 loads (5.40 B vs 1.65 B), yet runs 5× faster. The register-blocked loop re-reads each weight row per pixel block, so it *does* more bookkeeping — but because ~95 % of those loads hit L1 and stream predictably, they are nearly free, whereas the naive version's far fewer loads are almost all expensive misses. The lesson lands hard: **runtime is governed by where the data comes from, not by how many instructions or loads you issue**. Instruction throughput rises ~9.7× (0.38 → 3.67 G instr/s), which is exactly the signature of a CPU that was previously stalled on memory ~90 % of the time and is now kept fed.

4. Comparison of the two profiles

Metric	Naive	Optimized	Change
Runtime (compute)	11.37 s	2.28 s	~5.0× faster
L1-dcache miss rate	96.17 %	5.45 %	17.6× lower
L1-dcache load misses	1.587×10^9	0.295×10^9	5.4× fewer
L1-dcache loads	1.650×10^9	5.404×10^9	3.3× more

Metric	Naive	Optimized	Change
Instructions	4.459×10^9	8.763×10^9	2.0× more
Instr. throughput	0.38 G/s	3.67 G/s	9.7× higher

Reasoning / why the gain makes sense. Both versions perform the identical $C_{out} \cdot C_{in} \cdot H \cdot W \approx 4.3 \times 10^9$ multiply-accumulates. The naive version stalls because nearly every input load (96 %) misses L1 and must wait on a cache line fetched from a distant address — and most of that 64-byte line is discarded before it is reused. The CPU is memory-bound: it retires only ~ 0.38 G instr/s. The optimized version touches the same data in long contiguous streams, so ~ 95 % of loads hit L1, the hardware prefetcher hides the remaining latency, and each fetched line feeds many MACs. Note the apparent paradox in the table: the optimized code issues **more** loads and **more** instructions, yet wins by 5×. That is the whole point — what costs time is not the *count* of loads but their *latency*. Trading 1.6 B mostly-missing loads for 5.4 B mostly-hitting loads is a huge net win because an L1 hit costs a few cycles while a miss to L2/L3/DRAM costs tens to hundreds. The speedup is therefore not from doing less work; it is from **feeding the same work to the CPU at the rate the memory system can actually sustain** — visible as the 9.7× jump in instruction throughput.

5. Approaches considered / dropped

- **Compiler flags only.** Raising optimization on the naive code helped little — the bottleneck is the access pattern, which the compiler cannot legally reshuffle. This is why both versions share identical flags (`-O3 -mavx2 -mfma`): to isolate the layout effect.
- **Small problem size.** At `64×64` the whole working set fits in cache and the two versions run within $\sim 1\times$ of each other (we measured ~ 60 ms vs ~ 58 ms). We kept the larger size precisely so the cache effect is observable — the contrast itself is part of the lesson.
- **Full GEMM tiling (blocking both pixels and channels).** Would push performance further toward a BLAS library, but it muddies the single, clearly-explainable insight (layout + reuse) that this assignment rewards, so I left it as future work.

Reproduction

```
bash run.sh          # compile (g++) + verify identical output + perf stat, all-in-one
# or, if the profiling host has perf but not g++:
bash build_portable.sh # build once on a machine with g++ -> bin/
bash perf.sh          # run on the perf host using the prebuilt bin/ binaries
```

All progress prints are gated behind a `VERBOSE` compile-time define (`-DVERBOSE=0`) so timing/profiling runs are free of I/O noise and stdout stays diff-clean. The numbers in this report were collected with the `-O3 -mavx2 -mfma` portable build.